

The Dynamic Sporadic Server Algorithm for Aperiodic Task Scheduling in EDF Systems

Oleg Kelner
Hochschule Hamm-Lippstadt
Lippstadt, Germany
oleg.kelner@stud.hshl.de

Abstract—In real-time embedded systems, the joint scheduling of hard periodic tasks and soft aperiodic requests remains a critical challenge. While periodic tasks ensure system stability, aperiodic requests often require rapid response times without compromising hard deadlines. This paper explores the Dynamic Sporadic Server (DSS) algorithm, an extension of the fixed-priority sporadic server adapted for Earliest Deadline First (EDF) environments. We define the problem of aperiodic responsiveness, establish the mathematical foundations for DSS schedulability, and provide a state-machine-based C++ implementation model. Finally, the practical utility of DSS is demonstrated through its application in multimedia control and robotic systems.

I. INTRODUCTION

Real-time systems are increasingly required to support a mixture of workloads within a single processor: periodic activities such as control loops and sensor sampling, and aperiodic or sporadic activities such as operator inputs, fault handling, or event-driven sensor updates. Periodic tasks are typically associated with hard deadlines, where a single missed deadline can compromise system safety or correctness. Aperiodic tasks, by contrast, usually carry soft deadlines: timely service is desirable for system responsiveness, but an occasional delay does not lead to catastrophic failure. The scheduling problem is therefore not merely to guarantee the hard periodic task set, but to do so while minimising the average response time of the aperiodic workload, since a real-time system that is technically correct but unresponsive to operator or sensor events is of limited practical value.

The mechanisms used to schedule periodic tasks fall broadly into two families: fixed-priority policies, such as Rate Monotonic (RM), where a task's priority is static and derived from its period, and dynamic-priority policies, such as Earliest Deadline First (EDF), where priority is recomputed continuously from each task's current absolute deadline. EDF is of particular interest because it is optimal among preemptive uniprocessor scheduling algorithms with respect to schedulability: any periodic task set with total utilisation $U \leq 1$ is feasible under EDF. This optimality, however, complicates the design of mechanisms for aperiodic service, since most early aperiodic-service algorithms were developed for the simpler, static priority ordering of fixed-priority systems and do not transfer directly to a deadline-driven environment.

The simplest treatment of aperiodic tasks is *background scheduling*, in which aperiodic requests execute only in the idle intervals left by periodic tasks. This approach imposes

no risk to the periodic guarantee but produces poor average response times whenever the periodic load is moderate to high, since aperiodic requests may be starved indefinitely behind periodic execution. The *Polling Server* (PS) [1] improves on this by reserving a fixed capacity C_s within every period T_s , scheduled with a priority appropriate to T_s , for the exclusive use of aperiodic requests. The weakness of PS is that this capacity is only available at the instant the period begins: if no aperiodic request is pending at that instant, the capacity is discarded, even if a request arrives moments later. This idling behaviour makes PS *bandwidth-wasting* rather than *bandwidth-preserving*.

The *Deferrable Server* (DS) [1] addresses this specific weakness by allowing the server to retain its capacity throughout the period instead of discarding it immediately, so that a late-arriving aperiodic request can still be served at the server's priority. This preserves the server's responsiveness but introduces a more subtle problem: because capacity can be deferred and then immediately replenished at the start of the following period, a worst-case sequence of requests allows the server to execute, in effect, up to twice its budget within an interval no larger than T_s . This so-called "double hit" or "back-to-back execution" anomaly inflates the interference experienced by lower-priority periodic tasks beyond what is assumed in standard RM schedulability analysis, and consequently requires a pessimistic correction to the periodic task set's response-time analysis to remain safe.

The *Sporadic Server* (SS) [2] was proposed specifically to eliminate this anomaly while retaining capacity-preservation. Rather than replenishing capacity unconditionally at fixed period boundaries, SS ties each replenishment to the instant at which the corresponding capacity was *consumed*, delayed by exactly one server period T_s . This rule guarantees that the server's interference on the periodic task set never exceeds what a periodic task of the same period and capacity would cause, restoring exact RM schedulability analysis without the pessimistic correction required by DS. SS is, however, formulated entirely in terms of fixed priority levels and active priority tracking, which has no direct counterpart in EDF, where a task's "priority" (its absolute deadline) changes continuously as time advances.

For EDF-scheduled systems, several dynamic-priority counterparts to these bandwidth-preserving servers have since been proposed. The *Total Bandwidth Server* (TBS) [3] assigns

each arriving aperiodic request a deadline computed from the server's reserved utilisation U_s and the deadline of the previous request, guaranteeing that the server never consumes more than its allotted bandwidth over any interval. The *Constant Bandwidth Server* (CBS) [4] refines this further by postponing the server's deadline whenever its budget is exhausted before the next replenishment, which provides strong temporal isolation between the server and the rest of the task set even in the presence of execution-time overruns. The *Dynamic Sporadic Server* (DSS) [3], the subject of this work, takes a different route: rather than computing deadlines from a bandwidth formula, it carries the SS replenishment rule itself over into EDF, reformulating "replenish C_s at T_s after consumption" in terms of deadlines rather than priority levels. Compared to TBS and CBS, DSS retains a closer structural relationship to its fixed-priority ancestor, which makes its behaviour and schedulability argument easier to relate directly to the well-understood SS analysis, at the cost of being somewhat less flexible in handling budget overruns than CBS's deadline-postponement mechanism. Compared to background and polling service, DSS is bandwidth-preserving by construction and therefore offers substantially better average response time for the same guaranteed periodic schedule.

II. PROBLEM STATEMENT

The central problem addressed by the DSS algorithm is to optimise the average response time of soft aperiodic requests under EDF, while guaranteeing that the hard periodic task set remains exactly as schedulable as it would be without the server present — that is, the server must behave, from the periodic tasks' perspective, indistinguishably from an ordinary periodic task of utilisation $U_s = C_s/T_s$.

This requirement is non-trivial precisely because EDF assigns priority through absolute deadlines rather than static levels. A naïve dynamic-priority server that simply restores its full capacity C_s and assigns itself a new deadline $d = t + T_s$ at every period boundary, irrespective of whether or when it actually executed, can "double hit" the system: if the server is granted a fresh deadline before its previous capacity has been fully consumed or accounted for, it can demand its budget twice within an interval shorter than T_s , exactly the anomaly that the original Sporadic Server was designed to prevent in the fixed-priority case. Under EDF, this excess demand manifests not as a priority-inversion problem but as an unaccounted increase in the server's effective utilisation within a bounded interval, which can cause periodic deadlines to be missed despite $U \leq 1$.

The challenge, therefore, is to define a replenishment rule for C_s that:

- 1) preserves the server's capacity from the start of its period until an aperiodic request actually arrives, so that responsiveness is not sacrificed to idling, as in PS;
- 2) replenishes consumed capacity no earlier than exactly one server period after it was consumed, so that the server's cumulative demand over any interval of length T_s never exceeds C_s , avoiding the DS anomaly; and

- 3) expresses this rule purely in terms of deadlines and consumption instants, since EDF has no notion of a static "active priority level" to track, unlike the original SS formulation.

A further subtlety is the interaction between the server's own deadline and the deadlines of the aperiodic requests it serves: since EDF priority follows directly from deadline, the rule used to assign and update the server's deadline at execution and replenishment time is the rule that determines temporal isolation, and must be proven correct independently of any particular periodic task set, in the same way the original SS proof is independent of the specific RM priority assignment used. Establishing this replenishment rule, and showing that it preserves both responsiveness and the periodic schedulability guarantee, is the technical core of the DSS algorithm and the focus of the remainder of this work.

III. MATHEMATICAL FOUNDATIONS

A. Necessary Mathematical Basics

A DSS is defined by two primary parameters: a capacity C_s (the maximum execution budget) and a period T_s . The server utilization is given by $U_s = C_s/T_s$.

B. Formal Description of the Topic

1) *Deadline Assignment*: In an EDF environment, the server is treated as a periodic task with a dynamic deadline. When the server transitions from an idle state to a ready state at time t , it is assigned an absolute deadline d such that:

$$d = t + T_s \quad (1)$$

This deadline also defines the next replenishment time.

2) *Schedulability Analysis*: The inclusion of a DSS does not compromise the optimality of the EDF scheduler. A set of periodic tasks with utilization U_p and a DSS is schedulable if and only if the total processor utilization does not exceed unity:

$$U_p + U_s \leq 1 \quad (2)$$

where $U_p = \sum_{i=1}^n \frac{C_i}{T_i}$. This theorem ensures that as long as the server does not exceed its allocated bandwidth over any interval of length T_s , the periodic tasks remain safe.

IV. THE DYNAMIC SPORADIC SERVER ALGORITHM

The operation of the DSS is governed by a state machine consisting of three states: IDLE, READY, and EXE.

A. State Transitions

- **IDLE to READY**: Occurs when an aperiodic request enters the system and $C_s > 0$, or when capacity becomes greater than zero due to a replenishment. At this instant t , the deadline and next replenishment time are set to $t + T_s$.
- **READY to EXE**: Occurs when the server has the highest priority (shortest deadline) and there are pending aperiodic requests.
- **EXE to READY**: Occurs if the server is preempted by a periodic task with a shorter deadline.

- **EXE to IDLE:** Occurs if the aperiodic request is completed or if C_s is exhausted. A replenishment for the consumed capacity is scheduled to occur exactly T_s units after the server last entered the READY state from IDLE.

B. Replenishment Rule

Unlike a periodic server, DSS capacity is not replenished at the start of a period. Instead, each consumed portion of capacity is replenished exactly T_s units of time after the server became active to serve that specific portion. This prevents the "double-hit" problem and ensures the server behaves like a periodic task with period T_s and execution time C_s over the long term.

V. RUNTIME AND OVERHEAD ANALYSIS

The computational overhead of the DSS algorithm primarily resides in the management of the replenishment queue and the deadline assignment logic.

A. Algorithm Complexity

The selection of the highest priority task in an EDF system with N tasks typically requires $O(\log N)$ with a priority queue or $O(N)$ with a simple array. In our implementation, the 'select_task' function performs a linear scan, resulting in $O(N)$ complexity per scheduling point.

The replenishment rule requires $O(1)$ time to schedule a new replenishment (appending to the circular buffer) and $O(1)$ time to process a replenishment when the timer expires. Compared to the Total Bandwidth Server (TBS), which only requires a single deadline update per request, the DSS carries a slightly higher memory overhead due to the replenishment queue, but this is justified by the superior temporal isolation it provides.

VI. IMPLEMENTATION MODEL

The Dynamic Sporadic Server was implemented using UPPAAL Timed Automata. To ensure compatibility with standard model checking and avoid the undecidability associated with arbitrary stopwatch stop-clocks, a discrete-time modeling approach was adopted.

A. Model Structure

The model consists of four primary automaton templates:

- **PeriodicTask:** Models hard periodic tasks that trigger at regular intervals and request execution with a specific absolute deadline.
- **DSS:** Implements the server logic, including dynamic deadline assignment ($d = t + T_s$) and consumption tracking.
- **Replenisher:** Manages a FIFO queue of replenishment events, restoring budget to the DSS exactly T_s units after consumption began.
- **Ticker:** A centralized clock that synchronizes the system state and invokes the EDF scheduler at each time step.

B. EDF Scheduler Logic

The scheduler is implemented as a C-style function within the UPPAAL declaration. At each time step, it evaluates all ready tasks and assigns the CPU to the task with the smallest absolute deadline. This ensures that the dynamic priority nature of EDF is correctly captured.

VII. APPLICATION REALIZATION

The DSS implementation was validated against a task set consisting of two hard periodic tasks and one DSS. Formal verification using UPPAAL's verifier confirmed that the system is deadlock-free and that all hard periodic tasks meet their deadlines even under heavy aperiodic load.

The model is particularly useful in hierarchical embedded systems where soft applications, such as multimedia stream control, coexist with hard hardware control loops [3]. In robot control, periodic tasks ensure stability via sensor acquisition and low-level control, while the DSS manages operator requests or path planning updates.

A. Performance Comparison

Simulation results indicate that while DSS performs better than background or polling service, it may be outperformed by the Total Bandwidth Server (TBS) or Improved Priority Exchange (IPE) in terms of average response time for very heavy aperiodic loads. However, DSS provides better temporal isolation and protection during overloads than TBS. It is a balanced choice for designers who need to trade off implementation complexity against responsiveness.

VIII. CONCLUSION

The Dynamic Sporadic Server is a robust algorithm for managing aperiodic tasks in dynamic priority systems. By utilizing a replenishment rule tied to actual consumption, it provides rapid responsiveness while maintaining the hard guarantees of the underlying EDF schedule. Its low implementation complexity makes it a practical candidate for real-world real-time operating systems.

REFERENCES

- [1] J. K. Strosnider, J. P. Lehoczky, and L. Sha, "The deferrable server algorithm for enhanced aperiodic responsiveness in hard real-time environments," *IEEE Transactions on Computers*, vol. 44, pp. 73–91, 1995. [Online]. Available: <https://ieeexplore.ieee.org/document/368008>
- [2] B. Sprunt, L. Sha, and J. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems 1989 1:1*, vol. 1, pp. 27–60, 1989. [Online]. Available: <https://link.springer.com/article/10.1007/BF02341920>
- [3] M. Spuri and G. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, pp. 179–210, 3 1996. [Online]. Available: <http://link.springer.com/10.1007/BF00360340>
- [4] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," *Proceedings - Real-Time Systems Symposium*, pp. 4–13, 1998. [Online]. Available: <https://ieeexplore.ieee.org/document/739726>